ImageLab Test 2: Long Polling Implementation and Comparison

Alex Guerra and Renee Banner CMPS 4191 October 2026

1. Summary

We extended test 1 ImageLab application with version based long polling and kept one-second short polling as the baseline. Both modes observe the same durable PostgreSQL job and the same single worker. We ran 31 measured runs across five conditions (A, B, C1, C2, C3). Long polling let the browser learn the final result much sooner in every condition (about 15 to 230 ms vs 55 to 1,075 ms). It reduced requests only when the job was quiet (condition A). Image processing itself did not get faster in either mode.

2. What was changed to make long polling work

Layer	Change
Database	Migration 000004 adds version (starts at 1, backfills old jobs), stage, variants_completed and simulation_step, each with CHECK constraints.
Data layer	Every status or progress change increments version in the same UPDATE statement. Updates only apply while a job is processing, so a finished job can never change again.
Notifier	A new in-process notifier keeps one channel per job. After a commit it closes that channel, which wakes every waiting handler.
Endpoint	GET /v1/jobs/{id}?after=N&wait=S returns the snapshot immediately if the client is behind or the job is done, holds the request while versions match, and returns 204 at the holding deadline after rechecking the database. Invalid parameters return 400, unknown or malformed ids return 404.
Worker	After each commit it logs the commit time and notifies. Progress is published after each real variant (thumbnail_ready, preview_ready, display_ready).
Server	The long request gets its own write deadline (hold plus grace), and held requests end at shutdown.
Browser	A mode selector, a shared render function, one outstanding request, a 30 s timeout for long polling, immediate continuation after a 204, Stop and Resume, and a Try again button that observes the same job without another upload.
Evidence	The browser records every observation GET;

Preventing a missed change. The handler subscribes, captures the current channel, and only then reads the job from PostgreSQL. The worker commits first and notifies afterwards. A change before the read is seen by the read; a change after it closes the channel the handler already holds. A notification only means "read the database again", so it never proves that processing succeeded.

3. Method

We ran each condition in both modes at least three times on one computer, an empty queue, one observer tab in the foreground, and alternating mode order. Short polling waits one second after each response and never overlaps; long polling sends its next request immediately. Detection delay is the browser's `Date.now()` at receipt of the terminal response minus the server's wall-clock log time taken just after the terminal commit. This is an estimate with a few milliseconds of scheduling jitter. C3 repeats C2 with 200 ms added latency from a Chrome DevTools custom profile.

Condition	Setup	Our prediction
A quiet job	30 s delay, no updates	Short polling is faster
B real progress	three real variants	Short polling is faster
C1 slow updates	update every 2 s for 10 s	Long polling is faster
C2 fast updates	update every 200 ms for 10 s	Long polling is faster
C3 fast with latency	C2 plus 200 ms	Long polling is faster

4. Results

Cond.	Requests short / long	Detection delay (ms) short / long	DB reads short / long	Versions missed short / long
A	32 / 7	56 / 20	32 / 11	3 / 0
B	3 / 6	545 / 15	3 / 9	3 / 0
C1	13 / 11	721 / 21	13 / 19	3 / 0
C2	13 / 56	773 / 20	13 / 109	44 / 0
C3	11 / 50	1,075 / 229	11 / 51	44 to 45 / 5 to 6

All runs reached the expected terminal version (6, 11 or 56) with no retrieval errors and no cancellations. In condition A every long-polling run saw exactly one 204 at the 20 s deadline and continued normally.

5. Interpretation

- **Where long polling helped.** The browser learned the result sooner: about 15 to 20 ms against 545 to 770 ms in B, C1 and C2. In A it avoided about 25 empty requests and about 20 database reads.
- **Where it did not.** With frequent updates it sent one request per update (56 in C2 against 13) and about eight times the database reads, because each held request reads the job at entry and after each wake-up. Fewer HTTP requests can therefore hide repeated queries, and in C2 there were more of both.
- **Faster processing?** No. Processing time was the same in both modes (about 1.3 s of real work plus the known artificial delay). Only the time to discover the result changed.
- **Skipped versions.** Short polling saw about 12 of 56 versions in C2 but still received the final one, because the API returns the current snapshot, not a history. Long polling saw all 56 in C2 but missed 5 or 6 in C3, where a 200 ms round trip is as long as the update interval. If every event had to be kept, a snapshot API would not be enough and an event log with replay would be needed.
- **Latency.** Long polling's delay rose from about 20 ms to about 229 ms, roughly one added round trip. Short polling rose to about 1,075 ms because its one-second wait adds to the latency.
- **After each long-poll response** the browser renders and sends the next request immediately after a 200 with a newer active version or a 204, and stops on completed or failed. A continuing response could instead deliver several updates over one connection and remove the per-update round trip that causes the C3 gaps.

6. Limitations

Short-polling delays cluster tightly inside each group because the poll clock starts at upload and jobs finish at nearly the same offset, so we sampled almost one phase of the one-second cycle; they are not a random 0 to 1,000 ms average. Database reads count only the job-table reads, not the image lookups. Notifications are in-process and one worker is used, so a killed job can remain in processing.

7. Recommendation

For ordinary ImageLab use (condition B), long polling is sufficient and clearly better: the result appears about 15 ms after it is committed instead of about 545 ms later, for 3 extra requests. Short polling remains acceptable for very short jobs or where the simplest server is wanted. For frequent updates over a slower path, long polling still wins on delay but skips intermediate versions, and a continuing response would be the next design to evaluate.